TRABAJO PRÁCTICO — SEMANA 6

Arquitecturas CNN: Comparación, Batch Normalization y Transfer Learning

Semana	6 — Arquitecturas de Convolutional Neural Networks
Modalidad	Individual
Entregable	Repositorio GitHub + Informe PDF + Demo en clase
Plazo	Domingo 04/10/26 – 23.59
Puntaje total	20 puntos
Framework	PyTorch o TensorFlow/Keras

1. Descripción General

En este trabajo los estudiantes implementarán, entrenarán y analizarán arquitecturas clásicas de redes neuronales convolucionales sobre un dataset de clasificación de imágenes médicas real. El trabajo integra los cuatro grandes temas de la semana: estructura de una CNN, arquitecturas clásicas, Batch Normalization y Transfer Learning.

El objetivo no es solo obtener buena precisión, sino comprender empíricamente el impacto de cada componente y decisión de diseño.

2. Dataset

Blood Cell Image Dataset (Kaggle)

Dataset público de imágenes microscópicas de células sanguíneas con 4 clases de clasificación.

Repositorio	https://www.kaggle.com/datasets/paultimothymooney/blood-cells
Clases	4: EOSINOPHIL, LYMPHOCYTE, MONOCYTE, NEUTROPHIL
Imágenes	~12,500 imágenes RGB de 320x240 px
Split	Train / Test ya provisto por el dataset
Licencia	CC0: Public Domain

Justificación: el dataset es de tamaño manejable, multiclase, con imágenes en color que aprovechan bien las capas convolucionales, y tiene aplicación real en diagnóstico médico asistido por IA.

3. Tareas Requeridas

Tarea 1 — Implementación desde cero de dos arquitecturas (6 pts)

Implementar y entrenar desde cero las siguientes dos arquitecturas adaptadas al dataset:

- LeNet-5 adaptado
  - Ajustar el tamaño de entrada a 64×64×3
  - Mantener la estructura original: 2 bloques conv-pool + 3 capas FC
  - Añadir una variante con Batch Normalization después de cada capa convolucional
- VGG-11 simplificado
  - Reducir a la mitad los filtros de cada bloque para que sea entrenable en CPU/Colab
  - Añadir una variante con Batch Normalization

Para cada arquitectura documentar: número total de parámetros, tiempo de entrenamiento por época, accuracy en test, y curvas de pérdida/accuracy. Comparar las 4 variantes (LeNet, LeNet+BN, VGG-11, VGG-11+BN) en una tabla resumen.

Tarea 2 — Análisis del efecto de Batch Normalization (4 pts)

Diseñar un experimento controlado donde la única variable sea la presencia o ausencia de Batch Normalization:

- Graficar las curvas de pérdida de entrenamiento y validación para ambas variantes de una misma arquitectura.
- Analizar la velocidad de convergencia: ¿cuántas épocas necesita cada variante para llegar al 80% de accuracy?
- Discutir el efecto observado en términos de Internal Covariate Shift (citar el paper original de Ioffe & Szegedy, 2015).
- Explorar si Batch Normalization permite usar una tasa de aprendizaje más alta sin desestabilizar el entrenamiento.

Tarea 3 — Transfer Learning con ResNet-18 o VGG-16 preentrenada (6 pts)

Usar un modelo preentrenado en ImageNet y aplicar Transfer Learning al dataset de células sanguíneas. El grupo debe comparar tres estrategias:

Estrategia	Descripción	Capas entrenables
Feature Extraction	Congelar todas las capas convolucionales, solo entrenar la capa FC final.	Solo FC final



Fine-tuning parcial	Congelar los primeros 2 bloques, entrenar los últimos bloques + FC.	Últimos bloques + FC
Fine-tuning total	Descongelar todo el modelo y entrenar con LR muy pequeño.	Todas las capas

Comparar las 3 estrategias entre sí y contra el mejor modelo entrenado desde cero (Tarea 1). Justificar cuál estrategia recomendarían en un contexto de datos médicos limitados.

Tarea 4 — Informe técnico y análisis crítico (4 pts)

Redactar un informe en PDF (máximo 8 páginas, sin contar referencias) que incluya:

- Introducción: motivación del problema y del dataset.
- Metodología: descripción de cada experimento, hiperparámetros usados, estrategia de preprocesamiento.
- Resultados: tablas comparativas, gráficas de curvas de entrenamiento, matriz de confusión del mejor modelo.
- Discusión: análisis crítico de los resultados, limitaciones encontradas y propuestas de mejora.
- Conclusiones y referencias (incluir papers originales de LeNet, AlexNet, VGG, ResNet y Batch Normalization).

---

4. Entregables

Todo el trabajo debe estar en un repositorio público de GitHub con la siguiente estructura:

```
dl-semana6-[apellidos]/
├── README.md
├── data/
│   └── download_data.sh
├── notebooks/
│   ├── 01_eda.ipynb
│   ├── 02_lenet_vgg.ipynb
```

```
│   └── 03_transfer.ipynb
├── src/
│   ├── models.py
│   ├── train.py
│   └── utils.py
├── results/
│   └── figures/
└── informe.pdf
```

El README.md debe incluir: descripción del proyecto, requisitos (requirements.txt o environment.yml), instrucciones de instalación y ejecución, y tabla resumen de resultados con los principales hallazgos. Todos los script deben estar ejecutados.

5. Rúbrica de Evaluación

Criterio	Descripción	Puntaje
Implementación CNN (T1)	Correcta implementación de LeNet y VGG-11, sin y con BN. Código limpio y reproducible.	**6 pts**
Análisis BN (T2)	Experimento controlado, gráficas claras, discusión fundamentada teóricamente.	**4 pts**
Transfer Learning (T3)	Las 3 estrategias implementadas y comparadas correctamente. Justificación sólida.	**6 pts**
Informe PDF (T4)	Redacción clara, resultados bien presentados, análisis crítico, referencias correctas.	**4 pts**
Repositorio GitHub	Estructura ordenada, README completo, código reproducible en Google Colab.	**Bonificación +1 pt**

6. Restricciones y Consideraciones

- No se permite el uso de código de terceros que resuelva directamente las tareas (Hugging Face Trainer completo, AutoML, etc.).
- Sí se permite usar torchvision.models o tf.keras.applications para cargar pesos preentrenados en la Tarea 3.
- El entrenamiento puede realizarse en Google Colab (GPU gratuita). Se recomienda guardar checkpoints.
- Todos los experimentos deben ser reproducibles: fijar semillas (torch.manual_seed / tf.random.set_seed).
- Las gráficas deben ser claras, con ejes etiquetados, leyenda y título.
- El informe debe citarse en formato APA o IEEE.

7. Papers de Referencia Obligatoria

- LeCun et al. (1998). Gradient-based learning applied to document recognition. Proceedings of the IEEE.
- Krizhevsky et al. (2012). ImageNet classification with deep convolutional neural networks. NeurIPS.
- Simonyan & Zisserman (2014). Very deep convolutional networks for large-scale image recognition. ICLR 2015.
- He et al. (2016). Deep residual learning for image recognition. CVPR.
- Ioffe & Szegedy (2015). Batch normalization: Accelerating deep network training. ICML.

Cualquier duda o consulta debe realizarse a través del delgado del curso.